Tema 4

Realización de consultas

Base de datos

Tech Hub <FP>

euroformac

ÍNDICE

Objetivos de la unidad 4
1. Introducción 5
2. Consultas para extraer información: La sentencia SELECT 6
2.1 Introducción a la sentencia SELECT 6
2.2 Ejemplos de uso de la sentencia SELECT 7
3. Selección, filtrado y ordenación de registros 10
3.1 Selección (filtrado de columnas) 10
3.1.1 Selección de todas las columnas de una tabla 10
3.1.2 Selección de columnas específicas de una tabla 10
3.1.3 Selección de columnas con alias 11
3.1.4 Selección de columnas con funciones 11
3.2 Filtrado de filas 12
3.2.1 Filtrar registros por valor en una columna 13
3.2.2 Filtrar registros por rango de valores 13
3.2.3 Filtrar registros por coincidencia parcial 13
3.2.4 Combinar condiciones con operadores lógicos 13
3.2.5 Utilizar subconsultas en la cláusula WHERE 14
3.3 Ordenación de filas 14
3.3.1 Ordenando por una sola columna 15
3.3.2 Ordenando por varias columnas 15
3.3.3 Ordenando por una expresión 15
3.3.4 Ordenando por una fecha 16
4. Operadores en SQL 22
4.1 Visión global de los operadores en SQL 22
4.2 Operadores de comparación, lógicos, aritméticos y de precedencia 23
4.2.1 Operadores de comparación 23
4.2.2 Operadores lógicos 24
4.2.3 Operadores aritméticos 24
4.2.4 Operadores de precedencia 25

5. Consultas de resumen y funciones de agregado 30
6. Agrupaciones de registros. Filtrado de las agrupaciones 35
6.1 Agrupaciones de registros 36
6.2 Filtrado de agrupaciones 38
7. Composiciones internas 43
8. Composiciones externas 45
8.1 Composición externa izquierda 45
8.2 Composición externa derecha 47
9. Subconsultas 52
9.1 Devolución de valores individuales 52
9.2 Devolución de listas de valores 53
9.3 Devolución de tuplas de valores 53
9.4 Subconsultas correlacionadas 54
10. Consultas jerárquicas 59
11. Consultas de operaciones de conjuntos: Unión, intersección, direferencia 63
11.1 Unión 63
11.2 Intersección 64
11.3 Diferencia 67
Resumen 74
Glosario 75
Bibliografía 81

Objetivos de la unidad

El objetivo de este tema es adentrarnos en la **sintaxis del lenguaje SQL** para la realización de consultas sobre bases de datos relacionales. Entre dichos objetivos, destacaremos:

- **Comprender el lenguaje SQL**: Comprender el lenguaje utilizado para interactuar con una base de datos relacional. SQL es un lenguaje de programación estándar utilizado para administrar y manipular datos en una base de datos relacional.
- **Realizar consultas básicas**: Aprender a realizar consultas básicas en una base de datos relacional, como la selección de datos de una sola tabla, la ordenación de resultados, y la filtración de datos.
- **Realizar consultas avanzadas**: Además de las consultas básicas, también aprenderemos a realizar consultas avanzadas como la combinación de datos de varias tablas, la agrupación y la realización de cálculos.
- **Optimizar las consultas**: Aprenderemos a optimizar las consultas para mejorar su rendimiento. La optimización de las consultas es importante para mejorar la eficiencia de las consultas y reducir el tiempo de respuesta.

Tech Hub <FP> euroformac

1. Introducción

SQL (Structured Query Language) es un lenguaje de programación utilizado para administrar y manipular datos en una base de datos relacional. Fue desarrollado por IBM en la década de 1970 como parte del sistema de **gestión de bases de datos relacionales (RDBMS)** llamado **System R.**

SQL permite a los usuarios interactuar con una base de datos de una manera estructurada y controlada, lo que garantiza que la información se almacene, recupere y modifique de manera consistente y precisa.

Se utiliza para realizar diversas operaciones en una base de datos como la creación de tablas, la inserción, actualización y eliminación de datos, la consulta y la recuperación de información, y la administración de permisos y seguridad.

Una de las características clave de SQL es su capacidad para realizar consultas complejas de datos.

Las consultas son utilizadas para recuperar información específica de una o más tablas en una base de datos.

Las consultas pueden ser tan simples como seleccionar una sola columna de una tabla, o tan complejas como combinar datos de varias tablas, realizar cálculos y aplicar filtros para obtener información relevante.

SQL es un **lenguaje declarativo,** lo que significa que los usuarios no necesitan preocuparse por los detalles de cómo se realiza una operación en la base de datos.

En su lugar, los usuarios simplemente declaran lo que quieren hacer y SQL se encarga de determinar la mejor manera de hacerlo, haciendo que SQL sea fácil de usar para usuarios no técnicos.

Además de su uso en bases de datos relacionales, SQL también se utiliza en una variedad de aplicaciones y tecnologías como el análisis de datos, la minería de datos, la inteligencia artificial y la nube. SQL es compatible con la mayoría de las plataformas de bases de datos y es ampliamente adoptado por empresas de todo el mundo.

2. Consultas para extraer información: La sentencia SELECT

2.1 Introducción a la sentencia SELECT

Una de las partes más importantes de SQL es la capacidad de realizar **consultas de datos**.

Las consultas de datos son utilizadas para recuperar información de una base de datos y pueden variar en complejidad, desde simples consultas de selección de datos hasta consultas complejas que involucran la combinación de datos de varias tablas, cálculos y agrupaciones.

La parte de SQL dedicada a las consultas de datos se conoce como **"sentencia SELECT"**. Esta sentencia es utilizada para recuperar información de una o más tablas en una base de datos.

El esqueleto de la sentencia SELECT es el siguiente:

```
SELECT [DISTINCT] <columnas>
  FROM <tabla> [AS alias]
  [WHERE <condiciones>]
  [GROUP BY <columnas>]
  [HAVING <condiciones_grupo>]
  [ORDER BY <columnas> [ASC|DESC]]
  [LIMIT <n> [OFFSET <m>]];
```

A continuación, explicamos cada elemento del "esqueleto" de la sentencia SELECT:

- **SELECT**
  Constituye el inicio de la consulta y especifica las columnas o expresiones que han de recuperarse.
- **[DISTINCT]**
  Actúa como modificador opcional para eliminar filas duplicadas en el conjunto de resultados.

Lista separada por comas que puede incluir nombres de columnas, funciones de agregación o expresiones.

- **FROM**
  Indica la tabla o vista de la que se extraen los datos; en caso de múltiples tablas, se emplean cláusulas JOIN.
- **[AS alias]**

  Permite asignar un nombre alternativo a la tabla o columna, facilitando la legibilidad de la consulta.
- **[WHERE]**

  Aplica un filtrado a nivel de fila, devolviendo únicamente aquellas que satisfacen la condición booleana especificada.
- **[GROUP BY]**

  Agrupa las filas según los valores de las columnas indicadas para, a continuación, aplicar funciones de agregación.
- **[HAVING <condiciones_grupo>]**

  Filtra los grupos generados por GROUP BY en función de una condición que se evalúa sobre valores agregados.
- **[ORDER BY [ASC|DESC]]**

  Ordena el conjunto final de resultados según las columnas indicadas; ASC (ascendente) es el valor por defecto, DESC invierte el orden.
- **[LIMIT [OFFSET]]**

  Restringe el número de filas devueltas (LIMIT) y, opcionalmente, omite las primeras m filas (OFFSET), útil para paginación.

2.2 Ejemplos de uso de la sentencia SELECT

La sintaxis básica de una sentencia SELECT es la siguiente:

SELECT columna1, columna2, ... FROM Tabla;

En esta sintaxis, **columna1**, **columna2**, etc. son los nombres de las columnas que se desean recuperar de la tabla especificada en **Tabla**. Como puede observarse, deben ir separadas por comas.

Si se desea recuperar todas las columnas de una tabla, se puede utilizar el carácter comodín * en lugar de enumerar las columnas individualmente. Es decir, la sentencia sería así:

```
SELECT * FROM Tabla;
```

Además de la selección básica de columnas, la sentencia **SELECT** permite realizar una variedad de operaciones, como la ordenación de resultados, la filtración de datos utilizando cláusulas **WHERE**, la agrupación de datos utilizando la cláusula **GROUP BY**, y la realización de cálculos utilizando funciones integradas como **SUM**, **COUNT** y **AVG**.

La sentencia **SELECT** también puede ser utilizada para combinar datos de varias tablas utilizando la cláusula **JOIN**. La cláusula **JOIN** permite unir datos de dos o más tablas basándose en una columna común.

La combinación de datos de varias tablas es una tarea común en la mayoría de las aplicaciones de bases de datos, y es esencial para la realización de consultas complejas.

La parte de SQL dedicada a las consultas de datos es fundamental para trabajar con bases de datos relacionales. El conocimiento de la sintaxis básica y las operaciones avanzadas de la sentencia **SELECT** es esencial para poder recuperar información relevante de una base de datos y realizar análisis de datos eficaces.

A continuación, veamos un ejemplo práctico de una extracción básica de datos con la sentencia SELECT

BeautyGlow es un centro de estética unisex donde se ofrecen servicios de manicura, depilación, arreglo de barbas, tratamientos faciales y venta de productos cosméticos. Para mantener actualizada su web y su sistema de gestión, el equipo directivo necesita extraer información básica de su base de datos sobre clientes, servicios y empleados.

Las tablas relevantes para este caso son:

- **Cliente** (id, nombre, email, teléfono, fecha_alta)
- **Servicio** (id, nombre, duración_minutos, precio)
- **Empleado** (id, nombre, especialidad)

Tienes que resolver los siguientes aspectos:

1. Mostrar **todas** las columnas de la tabla Cliente.
2. Listar el **nombre y precio** de cada servicio disponible.

3. Obtener el **id y el nombre** de todos los empleados.

Solución

1. Todas las columnas de la tabla Cliente

SELECT *

FROM Cliente;

2. Nombre y precio de cada servicio

SELECT nombre, precio

FROM Servicio;

3. ID y nombre de todos los empleados

SELECT id, nombre

FROM Empleado;

Con estas consultas básicas, el equipo de BeautyGlow podrá generar listados completos de clientes, servicios y empleados para su aplicación web o para informes internos.

3. Selección, filtrado y ordenación de registros

3.1 Selección (filtrado de columnas)

Como vimos en la introducción, la sentencia **SELECT** es una de las herramientas más básicas y poderosas de SQL, ya que permite seleccionar y recuperar información específica de una base de datos.

Una de las operaciones más comunes de **SELECT** es la selección de columnas. Esto se logra al especificar los nombres de las columnas que se desean seleccionar en la sentencia **SELECT**.

3.1.1 Selección de todas las columnas de una tabla

Para seleccionar todas las columnas de una tabla se puede utilizar el carácter comodín * en lugar de especificar los nombres de las columnas.

Por ejemplo, para seleccionar todas las columnas de una tabla llamada clientes, se puede utilizar la siguiente sentencia **SELECT**:

```
SELECT * FROM Clientes;
```

3.1.2 Selección de columnas específicas de una tabla

Para seleccionar columnas específicas de una tabla, **se deben especificar los nombres de las columnas separados por comas** en la sentencia **SELECT**.

Por ejemplo, para seleccionar solo las columnas CIF y Razón Social de la tabla clientes, se puede utilizar la siguiente sentencia SELECT:

```
SELECT CIF, RazonSocial FROM Clientes;
```

3.1.3 Selección de columnas con alias

Es posible asignar un alias a una columna utilizando la palabra clave **AS** en la sentencia **SELECT**. Los alias son útiles para renombrar columnas o para hacer que los resultados de la consulta sean más legibles.

Por ejemplo, para mejorar la legibilidad de la columna " RazonSocial" de la tabla clientes y asignarle el alias " Razón Social" , se puede utilizar la siguiente sentencia SELECT:

SELECT CIF, RazonSocial as 'Razón social' FROM Clientes;

Importante

Cuando vayamos a usar alias con espacios en blanco, es importante entrecomillar su contenido para que el intérprete de SQL sepa dónde empieza y termina el alias y no lo confunda con otra columna.

3.1.4 Selección de columnas con funciones

SQL tiene varias funciones integradas que se pueden utilizar en la sentencia **SELECT** para realizar operaciones matemáticas o de otro tipo en las columnas seleccionadas.

Por ejemplo, para seleccionar la suma de los valores de la columna " limiteCredito" de la tabla Clientes, se puede utilizar la siguiente sentencia **SELECT**:

SELECT SUM(limiteCredito) FROM Clientes;

La consulta anterior nos devolverá un resultado con una sola fila (registro) y una sola columna con la sumatoria del valor de la columna " limiteCredito" de todas las filas (registros) que existan en la tabla Clientes.

Obviamente, si queremos que el nombre de esa columna tenga un nombre más significativo, podríamos haber usado un alias:

SELECT SUM(limiteCredito) as 'Limite de crédito total' FROM Clientes;

3.2 Filtrado de filas

Definición

La **cláusula WHERE** es una de las cláusulas más importantes en SQL ya que permite filtrar los resultados de una consulta y obtener solo los datos que cumplan con una determinada condición.

La **cláusula WHERE** es una herramienta muy poderosa en SQL que permite filtrar los datos y obtener solo la información que se necesita en una consulta. Es importante conocer bien la sintaxis y las diferentes condiciones que se pueden utilizar en la cláusula WHERE para aprovechar al máximo el potencial de SQL.

La sintaxis básica de la cláusula WHERE es la siguiente:

SELECT columna1, columna2, ... FROM tabla WHERE condición;

Donde "columna1, columna2, ..." son las columnas que se desean seleccionar de la tabla, "tabla" es el nombre de la tabla que contiene los datos y "condición" es la expresión que se utiliza para filtrar los datos.

La condición puede ser una expresión simple o una expresión compleja que involucre operadores lógicos y de comparación. Algunos ejemplos de condiciones comunes son:

- **Igualdad**: WHERE columna = valor
- **Desigualdad**: WHERE columna != valor
- **Mayor que**: WHERE columna > valor
- **Mayor o igual que**: WHERE columna >= valor
- **Menor que:** WHERE columna < valor
- **Menor o igual que**: WHERE columna <= valor
- **Combinación de condiciones**: WHERE condición1 AND/OR condición2

A continuación, veremos algunos ejemplos de cómo utilizar la cláusula WHERE en SQL:

3.2.1 Filtrar registros por valor en una columna

SELECT * FROM usuarios WHERE nombre = 'Juan';

Este ejemplo selecciona todos los registros de la tabla "usuarios" donde el valor de la columna "nombre" es igual a "Juan".

3.2.2 Filtrar registros por rango de valores

SELECT * FROM productos WHERE precio BETWEEN 10 AND 50;

Este ejemplo selecciona todos los registros de la tabla "productos" donde el valor de la columna "precio" está entre 10 y 50.

3.2.3 Filtrar registros por coincidencia parcial

SELECT * FROM empleados WHERE nombre LIKE 'Ana%';

Este ejemplo selecciona todos los registros de la tabla "empleados" donde el valor de la columna "nombre" comienza con "Ana".

SELECT * FROM empleados WHERE nombre LIKE '%Ana%';

Este ejemplo selecciona todos los registros de la tabla "empleados" donde el valor de la columna "nombre" contiene con "Ana".

SELECT * FROM empleados WHERE nombre LIKE '%Ana';

Este ejemplo selecciona todos los registros de la tabla "empleados" donde el valor de la columna "nombre" finaliza con "Ana".

3.2.4 Combinar condiciones con operadores lógicos

SELECT * FROM ventas WHERE fecha BETWEEN '2023-01-01' AND '2023-02-28' AND total > 100;

Este ejemplo selecciona todos los registros de la tabla "ventas" donde la fecha está entre el 1 de enero de 2023 y el 28 de febrero de 2023 y el valor de la columna "total" es mayor a 100.

3.2.5 Utilizar subconsultas en la cláusula WHERE

SELECT * FROM clientes WHERE id IN (SELECT id_cliente FROM pedidos WHERE fecha > '2023-01-01');

Este ejemplo selecciona todos los registros de la tabla "clientes" donde el valor de la columna "id" está incluido en el resultado de una subconsulta que selecciona todos los valores de la columna "id_cliente" de la tabla "pedidos" donde la fecha es posterior al 1 de enero de 2023.

3.3 Ordenación de filas

La ordenación de filas o registros se realiza utilizando la **cláusula ORDER BY.**

La cláusula **ORDER BY** en SQL se utiliza para ordenar los resultados de una consulta por uno o más campos en orden ascendente o descendente.

Importante

La sintaxis básica de la cláusula ORDER BY es la siguiente:

SELECT columna1, columna2, ...

FROM tabla

WHERE condición

ORDER BY columna1 [ASC|DESC], columna2 [ASC|DESC], ...

Donde:

- **Columna1, columna2**, ... son los nombres de las columnas que se desean seleccionar.
- **Tabla** es el nombre de la tabla en la que se desea hacer la consulta.
- **Condición** es la condición que se desea aplicar a los datos.
- **ASC** especifica orden ascendente (de menor a mayor).
- **DESC** especifica orden descendente (de mayor a menor).

Cabe decir, que no es imprescindible que antes de una cláusula **ORDER BY**, haya una cláusula **WHERE**. Podría darse el caso en el que necesitemos ordenar la totalidad de los registros de una tabla, sin filtrar ninguno mediante la cláusula **WHERE**.

En este caso, la sintaxis de la consulta sería:

SELECT columna1, columna2, ...

FROM tabla

ORDER BY columna1 [ASC|DESC], columna2 [ASC|DESC], ...

Aunque al igual que ocurría con la cláusula WHERE, es imposible incluir en este manual la totalidad de combinaciones posibles. A continuación, veremos algunos ejemplos de aplicación de la cláusula **ORDER BY.**

3.3.1 Ordenando por una sola columna

Supongamos que tenemos una tabla "empleados" con las columnas "id", "nombre" y "salario", y queremos seleccionar todos los empleados ordenados por salario en orden descendente. La consulta SQL sería:

SELECT * FROM empleados ORDER BY salario DESC;

3.3.2 Ordenando por varias columnas

Supongamos que queremos ordenar los empleados por salario en orden descendente y por nombre en orden ascendente. La consulta SQL sería:

SELECT * FROM empleados ORDER BY salario DESC, nombre ASC;

3.3.3 Ordenando por una expresión

Supongamos que queremos ordenar los empleados por el salario multiplicado por 12 (para obtener el salario anual) en orden descendente. La consulta SQL sería:

SELECT *, salario*12 as salario_anual FROM empleados ORDER BY salario_anual DESC;

3.3.4 Ordenando por una fecha

Supongamos que tenemos una tabla "ventas" con las columnas "id", "fecha" y "importe", y queremos seleccionar todas las ventas ordenadas por fecha en orden ascendente. La consulta SQL sería:

```
SELECT * FROM ventas ORDER BY fecha ASC;
```

Siguiendo con el ejemplo práctico propuesto, veamos cómo es la selección, filtrado y ordenación de registros

BeautyGlow necesita generar diversos informes sobre las citas atendidas durante junio de 2025. Para ello, dispone de las siguientes tablas:

- **Cliente** (id, nombre, email, teléfono, fecha_alta)
- **Servicio** (id, nombre, duración_minutos, precio)
- **Empleado** (id, nombre, especialidad)
- **Cita** (id, cliente_id, servicio_id, empleado_id, fecha, hora, estado)

Se desea extraer información concreta a partir de consultas SQL que demuestren operaciones de selección de columnas, filtrado de filas y ordenación de resultados.

Cuestiones a resolver

1. **Selección (filtrado de columnas)**

1.-Obtener todas las columnas de la tabla Cita.

2.-Listar únicamente los campos cliente_id, servicio_id, fecha y hora de cada cita.

3.-Mostrar la fecha como Fecha_Cita, la hora como Hora_Cita, el nombre del cliente y el nombre del servicio.

4.-Calcular la suma total de la duración (en minutos) de todas las citas de “Arreglo de barba” en junio de 2025.

2. **Filtrado de filas**

1.-Listar todas las citas cuyo servicio sea “Depilación integral”.

2.-Mostrar las citas agendadas entre el 10 y el 20 de junio de 2025 (ambos inclusive).

3.-Obtener las citas asignadas a empleados cuyo nombre comience por "Ana".

4.-Identificar los clientes que realizaron más de 3 citas en junio de 2025.

3. Ordenación de registros

1.-Ordenar todas las citas de junio de 2025 por fecha ascendente.

2.-Ordenar las mismas citas por servicio_id ascendente y hora descendente.

3.-Ordenar las citas según la marca temporal completa (fecha + hora) en orden descendente, empleando una expresión.

4.-Obtener las cinco citas más recientes de junio de 2025, ordenadas por fecha de forma descendente.

Solución

1. Todas las columnas de la tabla Cita

```
SELECT *
FROM Cita
WHERE fecha BETWEEN '2025-06-01' AND '2025-06-30';
```

2. Columnas específicas: cliente_id, servicio_id, fecha, hora

```
SELECT cliente_id, servicio_id, fecha, hora
FROM Cita
WHERE fecha BETWEEN '2025-06-01' AND '2025-06-30';
```

3. Alias y JOIN para mostrar nombres descriptivos

```
SELECT
  c.fecha      AS Fecha_Cita,
  c.hora       AS Hora_Cita,
  cl.nombre    AS Nombre_Cliente,
  s.nombre     AS Nombre_Servicio
```

```
FROM Cita AS c
JOIN Cliente    AS cl ON c.cliente_id  = cl.id
JOIN Servicio   AS s  ON c.servicio_id = s.id
WHERE c.fecha BETWEEN '2025-06-01' AND '2025-06-30';
```

4. Suma de duración de las citas de “Arreglo de barba”

```
SELECT
  SUM(s.duración_minutos) AS Duracion_Total_Arreglo_Barba
FROM Cita AS c
JOIN Servicio AS s ON c.servicio_id = s.id
WHERE s.nombre = 'Arreglo de barba'
  AND c.fecha BETWEEN '2025-06-01' AND '2025-06-30';
```

5. Filtrar citas de “Depilación integral”

```
SELECT *
FROM Cita AS c
JOIN Servicio AS s ON c.servicio_id = s.id
WHERE s.nombre = 'Depilación integral'
  AND c.fecha BETWEEN '2025-06-01' AND '2025-06-30';
```

6. Filtrar por rango de fechas (10–20 junio 2025)

```
SELECT *
FROM Cita
WHERE fecha BETWEEN '2025-06-10' AND '2025-06-20';
```

7. Filtrar por coincidencia parcial en empleado

```
SELECT *
FROM Cita AS c
JOIN Empleado AS e ON c.empleado_id = e.id
WHERE e.nombre LIKE 'Ana%';
```

8. Subconsulta: clientes con más de 3 citas en junio 2025

```
SELECT *
FROM Cliente AS cl
WHERE cl.id IN (
```

```
  SELECT c.cliente_id
  FROM Cita AS c
  WHERE c.fecha BETWEEN '2025-06-01' AND '2025-06-30'
  GROUP BY c.cliente_id
  HAVING COUNT(*) > 3
);
```

9. Ordenar citas por fecha ascendente

```
SELECT *
FROM Cita
WHERE fecha BETWEEN '2025-06-01' AND '2025-06-30'
ORDER BY fecha ASC;
```

10. Ordenar por servicio_id asc y hora desc

```
SELECT *
FROM Cita
WHERE fecha BETWEEN '2025-06-01' AND '2025-06-30'
ORDER BY servicio_id ASC, hora DESC;
```

11. Ordenar por marca temporal (fecha + hora) desc

```
SELECT *,
  TIMESTAMP(fecha, hora) AS FechaHora
FROM Cita
WHERE fecha BETWEEN '2025-06-01' AND '2025-06-30'
ORDER BY TIMESTAMP(fecha, hora) DESC;
```

12. Cinco citas más recientes de junio 2025

```
SELECT *
FROM Cita
```

```
WHERE fecha BETWEEN '2025-06-01' AND '2025-06-30'
ORDER BY fecha DESC
LIMIT 5;
```

4. Operadores en SQL

4.1 Visión global de los operadores en SQL

Definición

Los **operadores** en SQL son símbolos y palabras clave que se utilizan para realizar diversas operaciones en las consultas y manipulación de datos en bases de datos relacionales.

Los operadores se pueden clasificar en varias categorías tales como:

- **Operadores aritméticos**: Se utilizan para realizar operaciones matemáticas en las columnas numéricas de una tabla. Los operadores aritméticos incluyen + (suma), - (resta), * (multiplicación) y / (división).
- **Operadores de comparación**: Se utilizan para comparar valores en las columnas de una tabla y devolver verdadero o falso. Los operadores de comparación incluyen = (igual a), <> o != (no igual a), < (menor que), > (mayor que), <= (menor o igual que) y >= (mayor o igual que).
- **Operadores lógicos**: Se utilizan para combinar múltiples condiciones y devolver verdadero o falso. Los operadores lógicos incluyen AND (y), OR (o) y NOT (no).
- **Operadores de cadena**: Se utilizan para manipular valores de cadena (texto) en las columnas de una tabla. Los operadores de cadena incluyen CONCAT (concatenar), LIKE (buscar patrón), SUBSTRING (extraer una parte de la cadena) y LENGTH (obtener la longitud de la cadena).
- **Operadores de agrupación**: Se utilizan para agrupar filas o registros de datos en una tabla en función de una o varias columnas. Los operadores de agrupación incluyen GROUP BY (agrupar por) y HAVING (tener).

Es importante conocer y comprender los diferentes operadores en SQL para poder escribir consultas efectivas y manipular los datos de manera eficiente.

A continuación, detallaremos los operadores aritméticos, de comparación y lógicos.

En cuanto a los operadores de agrupación se explicarán en este mismo tema, y los operadores de cadena los abordaremos en temas posteriores.

4.2 Operadores de comparación, lógicos, aritméticos y de precedencia

4.2.1 Operadores de comparación

Importante

Los operadores de comparación se utilizan para comparar dos valores y devolver un resultado de verdadero o falso.

Los operadores de comparación en SQL incluyen:

- **Igual a: =**
- **Diferente a: !=**
- **Mayor que: >**
- **Mayor o igual que: >=**
- **Menor que: <**
- **Menor o igual que: <=**

Algunos ejemplos:

- Filtraremos aquellos registros de la tabla producto en el que el precio sea mayor a 10:

  SELECT * FROM productos WHERE precio > 10;

- Filtraremos aquellos registros de la tabla producto en el que el precio sea menor o igual a 10:

  SELECT * FROM productos WHERE precio <= 10;

- Filtraremos aquellos registros de la tabla producto en el que el precio sea distinto 10:

  SELECT * FROM productos WHERE precio != 10;

4.2.2 Operadores lógicos

Importante

Los operadores lógicos se utilizan para combinar expresiones lógicas y devolver un resultado de verdadero o falso.

Los operadores lógicos en SQL incluyen:

- **AND**: Devuelve verdadero si ambas expresiones son verdaderas.
- **OR**: Devuelve verdadero si al menos una de las expresiones es verdadera.
- **NOT**: Invierte el resultado de una expresión lógica.

Algunos ejemplos:

- Seleccionaremos aquellos productos en los que el precio sea superior a 10 y haya stock:

  SELECT * FROM productos WHERE (precio > 10) AND (stock > 0);

- Seleccionaremos aquellos productos en los que el precio sea superior a 10 o haya stock:

  SELECT * FROM productos WHERE (precio > 10) OR (stock > 0);

- Seleccionaremos aquellos productos en los que el precio NO sea superior a 10 usando el operador NOT:

  SELECT * FROM productos WHERE NOT (precio > 10);

4.2.3 Operadores aritméticos

Importante

Los operadores aritméticos se utilizan para realizar operaciones matemáticas en valores numéricos.

Los operadores aritméticos en SQL incluyen:

- **Suma**: +
- **Resta**: -
- **Multiplicación**: *
- **División**: /
- **Módulo**: % (devuelve el resto de una división)

Algunos ejemplos:

- Obtendremos la base imponible de la venta multiplicando el precio por la cantidad, de la factura 11:

  SELECT (precio * cantidad) AS "Base Imponible"

  FROM ventas

  WHERE NumeroFactura=11;

- Obtendremos el IVA a partir del importe total de venta multiplicándolo por 21 y dividiéndolo entre 100, de la factura 11:

  SELECT (precio * cantidad) AS "Base Imponible",

  (precio * cantidad) * (21/100) as IVA

  FROM ventas

  WHERE NumeroFactura=11;

4.2.4 Operadores de precedencia

Importante

La **precedencia de los operadores** en SQL consiste en aplicar la misma regla que en las matemáticas: primero se realizan las operaciones con mayor precedencia y luego las de menor precedencia.

En SQL, la precedencia de los operadores es la siguiente (de mayor a menor precedencia):

- **Paréntesis** ( )
- **Multiplicación** (*)
- **División** (/)
- **Módulo** (%)
- **Suma** (+)
- **Resta** (-)
- **Comparación** (>, >=, <, <=, =, !=)
- **NOT**
- **AND**
- **OR**

Es importante tener en cuenta la precedencia de los operadores al escribir consultas SQL complejas, ya que puede afectar el resultado de la consulta si los paréntesis no se utilizan correctamente para agrupar las expresiones lógicas.

Conozcamos cómo es el uso de operadores en SQL en nuestro ejemplo práctico

BeautyGlow no solo ofrece servicios estéticos, sino también una línea de productos cosméticos que gestiona en un inventario y vende a sus clientes. Para optimizar el análisis de ventas y stock, se necesita aplicar diversos operadores en SQL sobre las tablas:

- **Producto** (id, nombre, precio, stock)
- **Venta** (id, producto_id, cliente_id, cantidad, fecha)
- **Servicio** (id, nombre, duración_minutos, precio)

Cuestiones a resolver

1. **Comparación**
    - Listar los servicios cuyo precio sea mayor que 50 € y cuya duración sea menor o igual a 30 minutos.
    - Mostrar los productos con precio entre 10 € y 50 € (inclusive).
    - Obtener los productos cuyo precio sea distinto de 25 €.

2. **Lógicos**
   - Seleccionar las ventas en las que la cantidad vendida sea mayor que 2 AND el precio del producto sea mayor que 20 €.
   - Listar los productos que o bien no tienen existencias (stock = 0) OR tienen un precio superior a 100 €.
3. **Aritméticos**
   - Calcular, para cada venta, el importe total como cantidad * precio.
   - Filtrar las ventas cuyo importe total sea mayor que 100 €.
   - Identificar las ventas en las que la cantidad sea impar (usando el operador módulo %).
4. **Precedencia**
   - Mostrar, para cada venta, dos cálculos distintos sin y con paréntesis, y observar la diferencia:
   - 1.-cantidad + precio * 2
   - 2.-(cantidad + precio) * 2

Solución

1.1.-Servicios con precio > 50€ y duración ≤ 30 min

```
SELECT *
FROM Servicio
WHERE precio > 50
  AND duración_minutos <= 30;
```

1.2 Productos con precio entre 10€ y 50€

```
SELECT *
FROM Producto
WHERE precio BETWEEN 10 AND 50;
```

1.3 Productos con precio distinto de 25€

```
SELECT *
FROM Producto
WHERE precio != 25;
```

2.1 Ventas con cantidad > 2 AND precio producto > 20€

```
SELECT v.*
FROM Venta AS v
JOIN Producto AS p ON v.producto_id = p.id
WHERE v.cantidad > 2
  AND p.precio > 20;
```

2.2 Productos sin existencias OR con precio > 100€

```
SELECT *
FROM Producto
WHERE stock = 0
   OR precio > 100;
```

3.1 Importe total de cada venta (arithmetic)

```
SELECT
  v.id,
  v.cantidad * p.precio AS Importe_Total
FROM Venta AS v
JOIN Producto AS p ON v.producto_id = p.id;
```

3.2 Filtrar ventas con Importe_Total > 100€

```
SELECT
  v.id,
  v.cantidad * p.precio AS Importe_Total
```

```
FROM Venta AS v
JOIN Producto AS p ON v.producto_id = p.id
WHERE v.cantidad * p.precio > 100;
```

3.3 Ventas con cantidad impar (módulo)

```
SELECT *
FROM Venta
WHERE cantidad % 2 = 1;
```

4.1 Precedencia sin paréntesis

```
SELECT
  id,
  cantidad + precio * 2 AS Calc_Sin_Parentesis
FROM (
  SELECT v.id, v.cantidad, p.precio
  FROM Venta AS v
  JOIN Producto AS p ON v.producto_id = p.id
) AS sub;
```

4.2 Precedencia con paréntesis

```
SELECT
  id,
  (cantidad + precio) * 2 AS Calc_Con_Parentesis
FROM (
  SELECT v.id, v.cantidad, p.precio
  FROM Venta AS v
  JOIN Producto AS p ON v.producto_id = p.id
) AS sub;
```

5. Consultas de resumen y funciones de agregado

Definición

Una **consulta de resumen** en SQL es una consulta que se utiliza para calcular una operación de resumen sobre un conjunto de valores en una o varias columnas de una tabla.

El **resultado de una consulta de resumen es un solo valor agregado** que resume el conjunto de datos seleccionados.

Por ejemplo, se puede utilizar una consulta de resumen para encontrar el número total de filas en una tabla, el valor máximo o mínimo en una columna, el promedio de valores en una columna, etc.

Las funciones de agregado son utilizadas en las consultas de resumen para realizar cálculos sobre un conjunto de valores en una columna y devolver un solo valor de resultado.

Estas funciones realizan operaciones matemáticas sobre las columnas de una tabla como sumar valores, encontrar el valor máximo o mínimo, contar el número de filas o registros, calcular el promedio, etc.

Entre las funciones de agregado más comunes en SQL se encuentran:

- **COUNT**: Contador de registros.
- **SUM**: Suma el valor de una columna.
- **AVG**: Calcula la media aritmética de una columna.
- **MAX**: Devuelve el valor máximo de una columna.
- **MIN**: Devuelve el valor mínimo de una columna.

Por lo tanto, una consulta de resumen utiliza una función de agregado para calcular una operación de resumen sobre un conjunto de datos en una o varias columnas de una tabla, y devuelve un solo valor agregado como resultado.

Veamos algunos ejemplos para describir mejor cada una de las funciones de agregado:

- **COUNT**(): Se utiliza para contar el número de filas en una tabla o el número de valores no nulos en una columna.

  Ejemplo: **SELECT COUNT(*) FROM clientes;**

- **SUM**(): Se utiliza para calcular la suma de los valores en una columna numérica.

  Ejemplo: **SELECT SUM(total_ventas) FROM ventas;**

- **AVG**(): Se utiliza para calcular el promedio de los valores en una columna numérica.

  Ejemplo: **SELECT AVG(edad) FROM clientes;**

- **MAX**(): Se utiliza para obtener el valor máximo en una columna.

  Ejemplo: **SELECT MAX(total_ventas) FROM ventas;**

- **MIN**(): Se utiliza para obtener el valor mínimo en una columna.

  Ejemplo: **SELECT MIN(edad) FROM clientes;**

Las funciones de agregado se pueden combinar con otras cláusulas SQL como **GROUP BY** y **HAVING**, para realizar cálculos agregados en conjuntos específicos de datos.

Importante

Las funciones de agregado son una parte importante de SQL y se utilizan para realizar cálculos agregados en los datos de una tabla.

Con la combinación adecuada de funciones de agregado y cláusulas SQL, se pueden realizar consultas de resumen complejas para analizar y manipular los datos de manera efectiva.

Veamos de forma práctica cómo se llevan a cabo las consultas de resumen y funciones de agregado

BeautyGlow desea obtener métricas clave sobre la actividad de junio de 2025 para valorar el rendimiento de sus servicios y clientes. Se trabajará sobre estas tablas:

- **Cita** (id, cliente_id, servicio_id, empleado_id, fecha, hora, estado)
- **Servicio** (id, nombre, duración_minutos, precio)
- **Cliente** (id, nombre, email, teléfono, fecha_alta)

Se pide calcular distintos resúmenes con funciones de agregado, tanto globales como agrupadas por servicio o cliente.

Cuestiones a resolver:

1. **Total de citas en junio 2025**
2. **Ingreso total por servicios** realizados en junio 2025
3. **Precio medio** de los servicios prestados en junio 2025
4. **Servicio con más citas** en junio 2025 y número de citas
5. **Número de citas por cliente**, mostrando solo aquellos clientes con más de 3 citas
6. **Cliente con mayor número de citas** en ese mes

Solución

1. Total de citas en junio 2025

```
SELECT
  COUNT(*) AS Total_Citas_Junio
FROM Cita
WHERE fecha BETWEEN '2025-06-01' AND '2025-06-30';
```

2. Ingreso total por servicios en junio 2025

```
SELECT
  SUM(s.precio) AS Ingreso_Total_Junio
FROM Cita AS c
JOIN Servicio AS s ON c.servicio_id = s.id
WHERE c.fecha BETWEEN '2025-06-01' AND '2025-06-30';
```

3. Precio medio de servicios en junio 2025

```
SELECT
  AVG(s.precio) AS Precio_Medio_Servicio
FROM Cita AS c
JOIN Servicio AS s ON c.servicio_id = s.id
WHERE c.fecha BETWEEN '2025-06-01' AND '2025-06-30';
```

4. Servicio con más citas en junio 2025

```
SELECT
  s.nombre        AS Servicio,
  COUNT(*)        AS Num_Citas
FROM Cita AS c
JOIN Servicio AS s ON c.servicio_id = s.id
WHERE c.fecha BETWEEN '2025-06-01' AND '2025-06-30'
GROUP BY s.nombre
ORDER BY Num_Citas DESC
LIMIT 1;
```

5. Número de citas por cliente (solo clientes con > 3 citas)

```
SELECT
  cl.nombre       AS Cliente,
  COUNT(*)        AS Total_Citas
FROM Cita AS c
JOIN Cliente AS cl ON c.cliente_id = cl.id
WHERE c.fecha BETWEEN '2025-06-01' AND '2025-06-30'
GROUP BY cl.nombre
HAVING COUNT(*) > 3
ORDER BY Total_Citas DESC;
```

6. Cliente con mayor número de citas en junio 2025

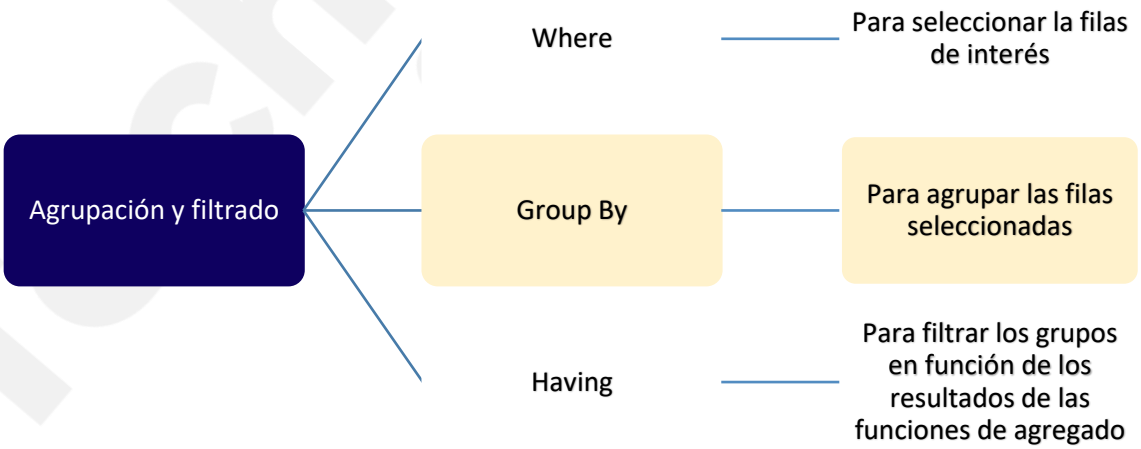
```
SELECT
  cl.nombre        AS Cliente_Top,
  COUNT(*)         AS Citas_Top
FROM Cita AS c
JOIN Cliente AS cl ON c.cliente_id = cl.id
WHERE c.fecha BETWEEN '2025-06-01' AND '2025-06-30'
GROUP BY cl.nombre
ORDER BY Citas_Top DESC
LIMIT 1;
```

6. Agrupaciones de registros. Filtrado de las agrupaciones

Las agrupaciones de registros permiten resumir grandes volúmenes de datos agrupando filas que comparten valores comunes en una o varias columnas. Gracias a la cláusula GROUP BY, podemos combinar registros en bloques lógicos y aplicar funciones de agregado (como SUM, COUNT, AVG, MAX o MIN) para obtener métricas útiles sobre cada grupo. Este mecanismo es fundamental cuando queremos calcular totales, promedios o conteos segmentados por categorías como tienda, periodo, cliente o cualquier otro atributo relevante.

Antes de agrupar, es habitual que necesitemos reducir el conjunto de datos original, seleccionando únicamente aquellas filas que cumplen unas condiciones determinadas. Para ello utilizamos la cláusula WHERE, que se ejecuta **antes** de la agrupación. WHERE filtra fila a fila: descarta aquellos registros que no cumplen la condición especificada, de modo que únicamente la porción de datos deseada llega al paso siguiente de agrupación.

Una vez aplicados los agrupamientos con GROUP BY, podemos necesitar refinar el conjunto de grupos resultante, quedándonos solo con aquellos que cumplen ciertos criterios basados en los valores agregados. Aquí es donde entra en juego la cláusula HAVING, que filtra **después** de la agrupación. A diferencia de WHERE, HAVING opera sobre los grupos y permite por ejemplo quedarnos únicamente con aquellos cuyo total de ventas supere un umbral o cuyo número de registros exceda un determinado valor.



6.1 Agrupaciones de registros

Las agrupaciones de registros en SQL se utilizan para combinar filas o registros en una tabla en grupos basados en valores comunes en una o varias columnas.

Las agrupaciones son útiles para realizar cálculos agregados en subconjuntos de datos y obtener información resumida.

La sintaxis básica de una consulta con agrupación es la siguiente:

```
SELECT columna1, columna2, función_de_agregado(columna3)
FROM tabla
GROUP BY columna1, columna2
```

En esta consulta se especifican las columnas que se van a seleccionar (columna1, columna2) y la función de agregado que se va a aplicar a la columna3.

Después, se utiliza la cláusula GROUP BY para agrupar las filas o registros basados en los valores de columna1 y columna2.

Importante

Es importante destacar que todas las columnas que se incluyan en la sentencia SELECT y que no sean de agregado, deben incluirse también en la cláusula GROUP BY.

Por ejemplo, la siguiente sentencia provocaría un error, ya que la columna3 no está incluida en la cláusula GROUP BY:

```
SELECT columna1, columna2, columna3, función_de_agregado(columna3)
FROM tabla
GROUP BY columna1, columna2
```

Vamos a ver un ejemplo:

Supongamos que tenemos una tabla "Franquicias" con información sobre las ventas de productos en diferentes tiendas:

Tienda	Número	Ventas
Cádiz	2023-01	1000
Sevilla	2023-01	1500
Cádiz	2023-02	700
Sevilla	2023-02	2000

Podemos utilizar una consulta con agrupación para obtener el total de ventas por tienda:

```
SELECT Tienda, SUM(Ventas)
FROM Franquicias
GROUP BY Tienda
```

El resultado de esta consulta sería:

Tienda	Ventas
Cádiz	1700
Sevilla	3500

El resultado de la consulta muestra las ventas totales por tienda, ya que se agrupan las filas o registros por el valor de la columna "Tienda" y se utiliza la función de agregado "SUM" para sumar las ventas.

Siguiendo con el ejemplo anterior, podríamos agrupar las ventas totales por periodo:

```
SELECT Periodo, SUM(Ventas)
FROM Franquicias
GROUP BY Periodo
```

El resultado de esta consulta sería:

Periodo	Ventas
2023-01	2500
2023-02	2700

6.2 Filtrado de agrupaciones

Además de la cláusula **GROUP BY**, se puede utilizar la cláusula **HAVING** para filtrar los grupos basados en ciertas condiciones.

Definición

La cláusula **HAVING** se utiliza en una consulta SQL para filtrar los resultados de una consulta con agrupación. La cláusula HAVING funciona de manera similar a la cláusula **WHERE**, pero se aplica después de que se hayan agrupado los resultados.

En una consulta con agrupación, la cláusula **GROUP BY** se utiliza para agrupar las filas en la tabla basadas en los valores de una o varias columnas.

Antes, en la sentencia **SELECT**, se utilizan funciones de agregado como **SUM**, **COUNT**, **AVG**, **MAX**, **MIN**, etc. para calcular valores resumidos sobre cada grupo.

Finalmente, la cláusula **HAVING** se utiliza para filtrar los grupos basados en ciertas condiciones que se aplican a los valores resumidos.

La sintaxis básica de una consulta con **HAVING** es la siguiente:

```
SELECT columna1, columna2, función_de_agregado(columna3)
FROM tabla
GROUP BY columna1, columna2
HAVING condición
```

En esta consulta se especifican las columnas que se van a seleccionar (columna1, columna2) y la función de agregado que se va a aplicar a la columna3.

Posteriormente, se utiliza la cláusula **GROUP BY** para agrupar las filas o registros basados en los valores de columna1 y columna2.

Finalmente, se utiliza la cláusula **HAVING** para filtrar los grupos basados en la condición especificada.

Veamos un ejemplo:

Volveremos a partir de la tabla "Franquicias" con información sobre las ventas de productos en diferentes tiendas:

Tienda	Periodo	Ventas
Cádiz	2023-01	1000
Sevilla	2023-01	1500
Cádiz	2023-02	700
Sevilla	2023-02	2000

Podemos utilizar una consulta con agrupación y HAVING para obtener el total de ventas por franquicia, pero solo para las franquicias que han tenido ventas totales superiores a 2000:

```
SELECT Tienda, SUM(Ventas)
FROM Franquicias
GROUP BY Tienda
HAVING SUM(Ventas) > 2000
```

El resultado de esta consulta sería:

Tienda	Ventas
Sevilla	3500

La consulta devuelve solo la franquicia "Sevilla", ya que es la única tienda que ha tenido ventas totales superiores a 2000.

Conozcamos cómo se lleva a cabo el análisis de métricas mensuales en BeautyGlow mediante agrupaciones

Como ya hemos visto en apartados anteriores, BeautyGlow es un centro de estética unisex que ofrece servicios de manicura, depilación, arreglo de barbas, tratamientos faciales y venta de productos cosméticos. Cada prestación queda registrada en la tabla Cita, que almacena el cliente, el servicio, el empleado, la fecha y el precio cobrado. La gerencia quiere obtener distintos informes resumidos para planificar turnos, promociones y stock.

Tablas relevantes

- **Servicio** (id, nombre, duración_minutos, precio_base)
- **Cliente** (id, nombre, email, fecha_alta)
- **Empleado** (id, nombre, especialidad)
- **Cita** (id, cliente_id, servicio_id, empleado_id, fecha, precio_real)
- **Venta** (id, cliente_id, producto_id, cantidad, fecha)

Preguntas a resolver

1. **Total de citas por servicio** durante marzo de 2025.
2. **Ingreso total** generado por cada servicio en marzo de 2025.
3. **Número medio de citas** por cliente en el mismo periodo.
4. **Servicios** que han superado 20 citas en marzo de 2025.

Soluciones

1. Total de citas por servicio en marzo de 2025

```
SELECT
  s.nombre        AS servicio,
  COUNT(*)        AS total_citas
FROM Cita c
JOIN Servicio s
  ON c.servicio_id = s.id
WHERE c.fecha
  BETWEEN '2025-03-01' AND '2025-03-31'
```

```
GROUP BY s.nombre
ORDER BY total_citas DESC;
```

2. Ingreso total por servicio en marzo de 2025

```
SELECT
  s.nombre        AS servicio,
  SUM(c.precio_real) AS ingreso_mensual
FROM Cita c
JOIN Servicio s
  ON c.servicio_id = s.id
WHERE c.fecha
  BETWEEN '2025-03-01' AND '2025-03-31'
GROUP BY s.nombre
ORDER BY ingreso_mensual DESC;
```

3. Número medio de citas por cliente en marzo de 2025

```
SELECT
  ROUND(AVG(ctes.citas_por_cliente), 2) AS media_citas_cliente
FROM (
  SELECT
    cliente_id,
    COUNT(*) AS citas_por_cliente
  FROM Cita
  WHERE fecha
    BETWEEN '2025-03-01' AND '2025-03-31'
  GROUP BY cliente_id
) AS ctes;
```

4. Servicios con más de 20 citas en marzo de 2025 (filtrado de agrupaciones)

```
SELECT
  s.nombre        AS servicio,
  COUNT(*)        AS total_citas
FROM Cita c
JOIN Servicio s
  ON c.servicio_id = s.id
WHERE c.fecha
  BETWEEN '2025-03-01' AND '2025-03-31'
GROUP BY s.nombre
HAVING COUNT(*) > 20
ORDER BY total_citas DESC;
```

7. Composiciones internas

Definición

La **composición interna**, también conocida como **INNER JOIN**, es un tipo de operación de unión en SQL que combina filas de dos o más tablas en función de una condición de igualdad entre las columnas de las tablas que se están uniendo.

En una composición interna, solo se devuelven las filas que tienen una coincidencia en ambas tablas. Las filas que no tienen una coincidencia se excluyen del resultado.

En SQL existen dos formas de expresar una composición interna: la sintaxis «implícita» (heredada de las primeras versiones del lenguaje) y la sintaxis «explícita», que es la recomendada hoy en día por su claridad y mantenibilidad. En la forma implícita se listan las tablas separadas por comas y la condición de unión va en la cláusula WHERE, por ejemplo:

- SELECT columnas
- FROM tabla1, tabla2
- WHERE tabla1.columna = tabla2.columna;

En la forma explícita, se usa INNER JOIN … ON, lo cual deja muy claro que la condición ON sirve para unir tablas, separándola de los filtros posteriores.

La sintaxis básica de una composición interna es la siguiente:

```
SELECT columnas
FROM tabla1
JOIN tabla2 ON tabla1.columna = tabla2.columna
```

En esta sintaxis se especifican las columnas que se van a seleccionar en la consulta, se indican las tablas que se van a unir (**tabla1 y tabla2**) y se especifica la condición de igualdad que se utilizará para unir las tablas (**tabla1.columna = tabla2.columna**).

Por ejemplo, si tenemos dos tablas, "clientes" y "pedidos", y queremos obtener una lista de todos los pedidos junto con la información del cliente correspondiente, podemos usar una composición interna para unir las dos tablas en función del número de cliente:

```
SELECT pedidos.*, clientes.nombre
FROM pedidos
JOIN clientes ON pedidos.cliente_id = clientes.id
```

En esta consulta, se seleccionan todas las columnas de la tabla "pedidos" y la columna "nombre" de la tabla "clientes".

Posteriormente, se unen las dos tablas en función de la condición de igualdad "pedidos.cliente_id = clientes.id".

El resultado sería una lista de todos los pedidos junto con el nombre del cliente correspondiente.

Importante

Es importante tener en cuenta que, en una composición interna, solo se devuelven las filas que tienen una coincidencia en ambas tablas.

Si hay filas en una tabla que no tienen una coincidencia en la otra tabla, esas filas se excluyen del resultado.

Aunque la mayoría de los JOINs se basan en igualdades (equi-joins), SQL permite unir tablas con cualquier condición relacional (theta-joins), por ejemplo, comparando fechas o rangos. No obstante, este tipo de uniones no equi suelen ser menos eficientes y se usan en casos muy concretos, como comparar registros temporales o superponer periodos.

Un INNER JOIN solo devuelve las filas que cumplen la condición de unión en ambas tablas. Si necesitamos incluir también las filas de una tabla, aunque no tengan correspondencia en la otra, deberemos usar composiciones externas (LEFT JOIN, RIGHT JOIN), que veremos en el siguiente apartado.

Para que los JOINs sean rápidos y escalables en bases de datos con muchos registros, es imprescindible que las columnas utilizadas en la condición de unión (normalmente las claves primarias y foráneas) estén correctamente indexadas. De lo contrario, el motor tendrá que hacer un escaneo completo de las tablas en cada unión, lo cual degrada drásticamente el rendimiento.

8. Composiciones externas

Definición

La **composición externa**, también conocida como **OUTER JOIN**, es un tipo de operación de unión en SQL que combina filas de dos o más tablas en función de una condición de igualdad entre las columnas de las tablas que se están uniendo.

A diferencia de la composición interna, **en una composición externa se devuelven todas las filas de al menos una de las tablas,** incluso si no tienen una coincidencia en la otra tabla.

Existen dos tipos de composiciones externas: la **composición externa izquierda** (**LEFT OUTER JOIN**) y la **composición externa derecha** (**RIGHT OUTER JOIN**).

8.1 Composición externa izquierda

En una composición externa izquierda se devuelven todas las filas o registros de la tabla izquierda y solo las filas de la tabla derecha que tienen una coincidencia en la tabla izquierda. Si no hay coincidencias en la tabla derecha, se devuelve NULL para las columnas correspondientes.

En una **composición externa izquierda** (LEFT OUTER JOIN), el motor de la base de datos recorre primero todos los registros de la tabla que aparece "a la izquierda" en la consulta y, para cada uno de ellos, busca las filas coincidentes en la tabla "derecha" según la condición ON.

Cada fila de la tabla izquierda formará parte del resultado, con independencia de que exista o no un registro coincidente en la tabla derecha. Esto permite, por ejemplo, listar todos los clientes, aunque no hayan realizado ningún pedido.

Cuando no se encuentra ninguna fila asociada en la tabla derecha, el motor rellena con NULL todas las columnas procedentes de esa tabla. De este modo, se conserva la integridad de la información de la izquierda, mientras se indica claramente que no hay datos relacionados.

Casos de uso típicos:

- Obtener usuarios y, si los tienen, sus últimos inicios de sesión: las filas de usuario sin actividad aparecerán igualmente, con campos de sesión a NULL.
- Listar productos y sus ventas: un producto sin ventas mantendrá sus datos, mostrando NULL en las columnas de ventas.

Dado que debe procesar todas las filas de la tabla izquierda y luego buscar coincidencias, es fundamental que la columna de unión esté indexada (idealmente, clave primaria en la izquierda y clave foránea indexada en la derecha). Esto evita escaneos completos y mejora la velocidad en conjuntos de datos grandes.

La sintaxis básica de una composición externa izquierda es la siguiente:

```
SELECT columnas
FROM tabla1
LEFT JOIN tabla2 ON tabla1.columna = tabla2.columna
```

En esta sintaxis

- Se especifican las columnas que se van a seleccionar en la consulta
- Se indica la tabla izquierda (tabla1)
- Se especifica la tabla derecha (tabla2)
- La condición de igualdad que se utilizará para unir las tablas (tabla1.columna = tabla2.columna).

El "**LEFT JOIN**" indica que se está realizando una composición externa izquierda.

Por ejemplo, si tenemos dos tablas, "clientes" y "pedidos", y queremos obtener una lista de todos los clientes junto con la información de sus pedidos correspondientes (si tienen algún pedido), podemos usar una composición externa izquierda para unir las dos tablas en función del número de cliente:

```
SELECT clientes.*, pedidos.fecha
FROM clientes
LEFT JOIN pedidos ON clientes.id = pedidos.cliente_id
```

En esta consulta

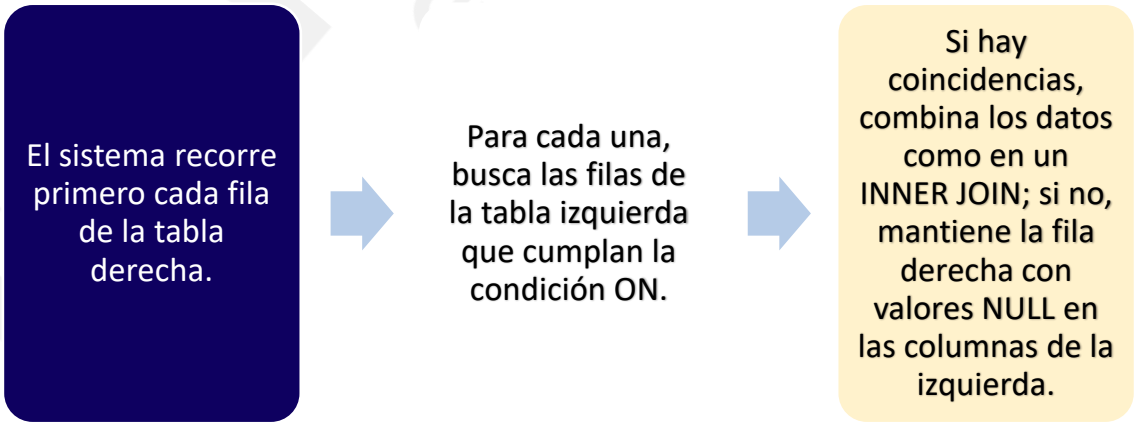
- Se seleccionan todas las columnas de la tabla "clientes" y la columna "fecha" de la tabla "pedidos".
- Se unen las dos tablas en función de la condición de igualdad "clientes.id = pedidos.cliente_id".

El "**LEFT JOIN**" indica que se deben devolver todas las filas o registros de la tabla "clientes", incluso si no tienen un pedido correspondiente. Si un cliente no tiene ningún pedido, se devolverá NULL para la columna "fecha".

8.2 Composición externa derecha

En una composición externa derecha se devuelven todas las filas o registros de la tabla derecha y solo las filas o registros de la tabla izquierda que tienen una coincidencia en la tabla derecha. Si no hay coincidencias en la tabla izquierda se devuelve NULL para las columnas correspondientes.

Todos los registros de la tabla derecha se incluyen siempre en el resultado, independientemente de que exista una fila coincidente en la tabla izquierda. Esto es útil cuando queremos asegurarnos de listar, por ejemplo, todos los productos de un catálogo (tabla derecha), incluso de aquellos que no han sido vendidos (tabla izquierda).



La sintaxis es similar a la composición externa izquierda, pero se utiliza "**RIGHT JOIN**" en lugar de "**LEFT JOIN**".

La sintaxis básica de una composición externa derecha es la siguiente:

```
SELECT columnas
FROM tabla1
RIGHT JOIN tabla2 ON tabla1.columna = tabla2.columna
```

En esta sintaxis

- Se especifican las columnas que se van a seleccionar en la consulta
- Se indica la tabla izquierda (tabla1)
- Se especifica la tabla derecha (tabla2)
- La condición de igualdad que se utilizará para unir las tablas (tabla1.columna = tabla2.columna).

El "**RIGHT JOIN**" indica que se está realizando una composición externa derecha.

Por ejemplo, si tenemos dos tablas, "clientes" y "pedidos", y queremos obtener una lista de todos los clientes junto con la información de sus pedidos correspondientes, podemos usar una composición externa derecha para unir las dos tablas en función del número de cliente:

```
SELECT clientes.*, pedidos.fecha
FROM clientes
RIGHT JOIN pedidos ON clientes.id = pedidos.cliente_id
```

En esta consulta

- Se seleccionan todas las columnas de la tabla "clientes" y la columna "fecha" de la tabla "pedidos".
- Se unen las dos tablas en función de la condición de igualdad "clientes.id = pedidos.cliente_id".

El "**RIGHT JOIN**" indica que se deben devolver todas las filas de la tabla "pedidos", incluso si no tienen un cliente correspondiente (en una base de dados relacional, normalmente no se dará este caso, pero para seguir con el

ejemplo de la composición externa izquierda, imaginemos que sí). Si un pedido no tiene ningún cliente, se devolverá NULL para las columnas de la tabla clientes.

Siguiendo con nuestro ejemplo veamos cómo se hace un listado de clientes y pedidos con composiciones externas en BeautyGlow

BeautyGlow quiere generar informes que incluyan tanto a los clientes sin actividad como a los pedidos huérfanos (sin cliente registrado) para detectar posibles errores de carga de datos. Para ello dispone de dos tablas:

- **clientes** (id, nombre, email)
- **pedidos** (id_pedido, cliente_id, fecha, total)

Ejemplo de contenido:

clientes

id	nombre	email
1	Ana	ana@beautyglow.es
2	Luis	luis@beautyglow.es
3	Marta	marta@beautyglow.es

pedidos

id_pedido	cliente_id	fecha	total
201	1	2025-06-10	45.00
202	2	2025-06-15	30.00
203	4	2025-06-20	60.00

Obs.: el pedido 203 tiene cliente_id = 4, que no existe en clientes.

Preguntas

1. Mediante LEFT JOIN, obtener la lista de todos los clientes junto con la fecha de sus pedidos (si no tienen, que aparezca NULL).
2. Mediante RIGHT JOIN, obtener la lista de todos los pedidos junto con el nombre del cliente (si no existe, que aparezca NULL).

Soluciones

1. **LEFT JOIN:** todos los clientes (incluso sin pedidos)

```
SELECT
  clientes.id,
  clientes.nombre,
  pedidos.fecha
FROM clientes
LEFT JOIN pedidos
  ON clientes.id = pedidos.cliente_id
ORDER BY clientes.id;
```

Resultado esperado:

id	nombre	fecha
1	Ana	2025-06-10
2	Luis	2025-06-15
3	Marta	NULL

2. **RIGHT JOIN:** todos los pedidos (incluso huérfanos)

```
SELECT
  pedidos.id_pedido,
  pedidos.fecha,
  clientes.nombre
FROM clientes
RIGHT JOIN pedidos
  ON clientes.id = pedidos.cliente_id
ORDER BY pedidos.id_pedido;
```

Resultado esperado:

id_pedido	fecha	nombre
201	2025-06-10	Ana
202	2025-06-15	Luis
203	2025-06-20	NULL

Interpretación

- Con **LEFT JOIN** se garantizan todos los clientes en el listado, incluso Marta (que no tiene pedidos).
- Con **RIGHT JOIN** se incluyen todos los pedidos, como el 203, destacando que su cliente_id no existe y, por tanto, nombre aparece NULL.

De esta forma, BeautyGlow puede detectar clientes inactivos y pedidos sin cliente asignado para corregir su base de datos y evitar pérdidas de información.

9. Subconsultas

Definición

Las **subconsultas** son consultas anidadas dentro de otras consultas en SQL.

Las subconsultas pueden ser utilizadas para devolver valores individuales, listas de valores o tuplas de valores, y también pueden ser correlacionadas.

9.1 Devolución de valores individuales

Importante

Una subconsulta que devuelve un solo valor se puede utilizar en cualquier lugar donde se espere una expresión, como en la cláusula SELECT o en una cláusula WHERE.

Por ejemplo, si queremos obtener el número de pedidos realizados por un cliente específico, podemos utilizar la siguiente subconsulta:

```
SELECT COUNT(*) FROM pedidos
WHERE cliente_id =
(SELECT id FROM clientes WHERE nombre = 'Filaturas, S.A.');
```

En esta consulta, la subconsulta "**SELECT id FROM clientes WHERE nombre = "Filaturas, S.A.**" devuelve el ID del cliente llamado "**Filaturas, S.A.**", que se utiliza como condición de igualdad en la cláusula WHERE de la consulta principal.

Otro ejemplo podría ser la devolución de la suma del importe total de los pedidos del cliente " **Filaturas, S.A."**

```
SELECT SUM(ImportePedido) FROM pedidos
WHERE cliente_id =
```

(SELECT id FROM clientes WHERE nombre = 'Filaturas, S.A.');

Al igual que antes, en esta consulta, la subconsulta "**SELECT id FROM clientes WHERE nombre = "Filaturas, S.A.**" devuelve el ID del cliente llamado **"Filaturas, S.A."**, que se utiliza como condición de igualdad en la cláusula WHERE de la consulta principal.

9.2 Devolución de listas de valores

Importante

Una subconsulta que devuelve una lista de valores puede ser utilizada en una cláusula WHERE con la palabra clave IN.

Por ejemplo, si queremos obtener una lista de los clientes que han realizado al menos un pedido, podemos utilizar la siguiente subconsulta:

```
SELECT nombre FROM clientes
WHERE id IN
(SELECT cliente_id FROM pedidos);
```

En esta consulta, la subconsulta **"SELECT cliente_id FROM pedidos"** devuelve una lista de identificadores de clientes que han realizado un pedido, que se utiliza en la cláusula **WHERE** de la consulta principal con la palabra clave **IN**.

9.3 Devolución de tuplas de valores

Una subconsulta que devuelve varias columnas —es decir, una *tupla* de valores— puede utilizarse en la cláusula WHERE mediante comparación de tuplas o con IN sobre varias columnas.

Por ejemplo, imagina una tabla **ventas** con los campos (cliente_id, producto_id, cantidad). Queremos obtener la lista de todas las combinaciones de cliente y producto cuya venta total supere 5 unidades. Para ello, usamos una subconsulta que agrupa por ambas columnas y devuelve tuplas (cliente_id, producto_id):

```
SELECT
  cliente_id,
  producto_id
FROM ventas
WHERE (cliente_id, producto_id) IN (
  SELECT
    cliente_id,
    producto_id
  FROM ventas
  GROUP BY cliente_id, producto_id
  HAVING SUM(cantidad) > 5
);
```

La subconsulta interior retorna **tuplas** (cliente_id, producto_id) en las que la suma de cantidad es mayor que 5.

En la consulta principal, el predicado (cliente_id, producto_id) IN (…) compara cada par de columnas de **ventas** con las tuplas devueltas, devolviendo sólo aquellas filas que coinciden con alguno de los pares del conjunto.

Así podemos filtrar combinaciones de varias columnas de forma sencilla y expresiva.

9.4 Subconsultas correlacionadas

Importante

Una subconsulta correlacionada utiliza valores de la consulta principal en la subconsulta.

Por ejemplo, si queremos obtener una lista de los clientes que han realizado más de un pedido, podemos utilizar la siguiente subconsulta correlacionada:

SELECT nombre FROM clientes

WHERE (SELECT COUNT(*) FROM pedidos WHERE pedidos.cliente_id = clientes.id) > 1;

En esta consulta, la subconsulta **"SELECT COUNT(*) FROM pedidos WHERE pedidos.cliente_id = clientres.id"** utiliza el valor de ID del cliente actual en la consulta principal para contar el número de pedidos realizados por ese cliente.

La cláusula **WHERE** de la consulta principal utiliza el resultado de la subconsulta para filtrar los clientes que han realizado más de un pedido.

Caso práctico resuelto: Análisis de clientes y ventas en BeautyGlow con subconsultas

BeautyGlow registra tanto los servicios (pedidos) realizados por sus clientes como las ventas de productos cosméticos. Dispone de estas tablas simplificadas:

- **clientes** (id, nombre, email)
- **pedidos** (id_pedido, cliente_id, fecha, total)
- **productos** (id_producto, nombre_producto, precio_unitario)
- **ventas** (id_venta, cliente_id, producto_id, cantidad)

Con estas tablas, queremos extraer información utilizando distintos tipos de subconsultas.

Preguntas

1. **Subconsulta escalar**: ¿Cuántos pedidos ha realizado el cliente cuya razón social es "Ana"?
2. **Subconsulta con IN**: Obtener el nombre de todos los clientes que han realizado al menos un pedido.
3. **Subconsulta con tuplas:** Listar las parejas (cliente_id, producto_id) para las que se han vendido más de 3 unidades en total.
4. **Subconsulta correlacionada**: Obtener los nombres de los clientes que han efectuado más de 2 pedidos.

Soluciones

1. Subconsulta escalar: número de pedidos de "Ana"

```
SELECT
 (SELECT COUNT(*)
  FROM pedidos
  WHERE cliente_id =
   (SELECT id
    FROM clientes
    WHERE nombre = 'Ana')
 ) AS num_pedidos_ana;
```

La subconsulta interna (SELECT id FROM clientes WHERE nombre = 'Ana') devuelve el id de Ana.

La subconsulta externa COUNT(*) cuenta los pedidos de ese cliente_id.

2. Subconsulta con IN: clientes con al menos un pedido

```
SELECT nombre
FROM clientes
WHERE id IN (
 SELECT cliente_id
 FROM pedidos
);
```

La subconsulta SELECT cliente_id FROM pedidos genera la lista de todos los clientes con pedidos.

El WHERE id IN (…) filtra la tabla clientes para quedarse solo con esos.

3. Subconsulta con tuplas: combinaciones cliente-producto con >3 unidades vendidas

```
SELECT
 cliente_id,
 producto_id
FROM ventas
WHERE (cliente_id, producto_id) IN (
```

```
  SELECT
   cliente_id,
   producto_id
  FROM ventas
  GROUP BY cliente_id, producto_id
  HAVING SUM(cantidad) > 3
);
```

- La subconsulta interior agrupa por (cliente_id, producto_id) y devuelve tuplas cuyo total supera 3.
- El predicado (cliente_id, producto_id) IN (…) compara cada fila de ventas contra esas tuplas.

4. Subconsulta correlacionada: clientes con más de 2 pedidos

```
SELECT nombre
FROM clientes
WHERE (
  SELECT COUNT(*)
  FROM pedidos
  WHERE pedidos.cliente_id = clientes.id
) > 2;
```

- La subconsulta SELECT COUNT(*) … WHERE pedidos.cliente_id = clientes.id se reevalúa por cada fila de clientes.
- La condición > 2 en el WHERE exterior filtra los clientes con más de dos pedidos.

Con este caso práctico, aplicamos:

- **Subconsulta escalar** (valor individual).
- **Subconsulta con IN** (lista de valores).
- **Subconsulta de tuplas** (varias columnas).
- **Subconsulta correlacionada** (dependiente de cada fila de la consulta principal).

Así BeautyGlow puede obtener métricas precisas sobre la actividad de sus clientes y optimizar ofertas y campañas.

Tech Hub <FP> euroformac

10. Consultas jerárquicas

Las consultas jerárquicas en SQL son aquellas que **permiten representar datos en forma de árbol o jerarquía,** en la que cada registro o fila puede tener uno o varios registros asociados que se encuentran en una relación padre-hijo.

Se trata de una estructura particular de base de datos, diferente al modelo relacional.

Las consultas jerárquicas son útiles en aplicaciones que requieren la visualización de datos en forma de árbol, como la organización de una empresa, los niveles de una categoría de productos, los componentes de un producto, entre otros.

En Oracle, se pueden realizar consultas jerárquicas utilizando la cláusula **CONNECT BY** en la cláusula **SELECT**. La sintaxis básica **de una consulta jerárquica es la siguiente:**

```
SELECT columnas
FROM tabla
WHERE condición
START WITH columna_inicio = valor_inicio;
CONNECT BY PRIOR columna_padre = columna_hijo
```

La cláusula **CONNECT BY** especifica cómo se relacionan los registros padre e hijo.

La expresión **"PRIOR columna_padre = columna_hijo"** indica que la columna_padre es la columna de la tabla que representa la relación padre-hijo, y la columna_hijo es la columna de la tabla que representa la relación hijo-padre.

La cláusula **START WITH** especifica el registro raíz de la jerarquía.

El valor_inicio es el valor de la columna_inicio que identifica el registro raíz.

Por ejemplo, si tenemos una tabla llamada "**departamentos**" que contiene la información de los departamentos de una empresa, donde cada departamento puede tener uno o varios subdepartamentos, podemos utilizar la siguiente consulta jerárquica para obtener la estructura completa de la jerarquía de departamentos:

```
SELECT nombre, nivel
FROM departamentos
START WITH id_departamento_padre IS NULL;
CONNECT BY PRIOR id_departamento = id_departamento_padre
```

En esta consulta, la cláusula **CONNECT BY** especifica que la columna "id_departamento" es la columna padre, y la columna "id_departamento_padre" es la columna hijo.

La cláusula **START WITH** especifica que el registro raíz es aquel que tiene un valor nulo en la columna "id_departamento_padre".

Importante

La consulta devuelve los nombres de los departamentos y el nivel en la jerarquía que ocupan. Los registros de nivel 1 corresponden a los departamentos de primer nivel, los registros de nivel 2 corresponden a los subdepartamentos, y así sucesivamente.

Caso práctico: Gestión de la jerarquía de empleados en BeautyGlow

BeautyGlow ha crecido y cuenta ahora con varios niveles de responsabilidad en su equipo. Para visualizar y analizar la estructura organizativa de la empresa, se ha diseñado la siguiente tabla empleados:

id	nombre	Puesto	manager_id
1	Ana	Directora General	NULL
2	Luis	Gerente de Sucursal	1
3	Marta	Gerente de Sucursal	1
4	Carlos	Estilista Senior	2
5	Lucía	Estilista Junior	2
6	Sara	Estilista Senior	3
7	Diego	Estilista Junior	6

- **manager_id** referencia al id del jefe directo de cada empleado.
- El nivel más alto (Ana) tiene manager_id = NULL.

Preguntas

1. **Jerarquía completa**: Listar todos los empleados con su nivel en la organización, mostrando una indentación visual en el nombre según su profundidad.
2. **Subordinados de un gerente:** Obtener la lista de todos los empleados que dependen (directa o indirectamente) de "Luis".

Soluciones

1. **Jerarquía completa con niveles e indentación**

```
SELECT
  LPAD(' ', 2*(LEVEL-1)) || nombre AS nombre_indentado,
  puesto,
  LEVEL                AS nivel
FROM empleados
START WITH manager_id IS NULL
CONNECT BY PRIOR id = manager_id
ORDER SIBLINGS BY nombre;
```

LEVEL: pseudo-columna de Oracle que indica la profundidad (1 = raíz).

LPAD(' ', 2*(LEVEL-1)) || nombre: añade dos espacios por cada nivel para indentar el nombre.

START WITH manager_id IS NULL: arranca en los jefes de más alto rango (Ana).

CONNECT BY PRIOR id = manager_id: define la relación padre → hijo (del id del jefe al manager_id del subordinado).

ORDER SIBLINGS BY nombre: ordena alfabéticamente hermanos en cada nivel.

Resultado esperado:

nombre_indentado	Puesto	nivel
Ana	Directora General	1
Luis	Gerente de Sucursal	2
Carlos	Estilista Senior	3
Lucía	Estilista Junior	3
Marta	Gerente de Sucursal	2
Sara	Estilista Senior	3

Diego	Estilista Junior	4

2. **Subordinados de "Luis" (toda la cadena)**

```
SELECT
  nombre,
  puesto,
  LEVEL          AS nivel_relativo
FROM empleados
START WITH nombre = 'Luis'
CONNECT BY PRIOR id = manager_id
ORDER SIBLINGS BY nombre;
```

- **START WITH nombre = 'Luis'**: arranca la jerarquía en Luis.
- Los niveles se cuentan desde Luis (nivel_relativo = 1 → Luis; 2 → sus subordinados directos; etc.).

Resultado esperado:

nombre	puesto	nivel_relativo
Luis	Gerente de Sucursal	1
Carlos	Estilista Senior	2
Lucía	Estilista Junior	2

Con estas consultas, BeautyGlow podrá:

- **Visualizar** toda la estructura de su plantilla, destacando la línea de mando.
- **Identificar** fácilmente todos los colaboradores bajo un responsable concreto, para asignarles formaciones, evaluar cargas de trabajo o comunicar decisiones.

11. Consultas de operaciones de conjuntos: Unión, intersección, direferencia

En SQL las consultas de operaciones de conjuntos permiten combinar o comparar conjuntos de resultados de dos o más consultas.

Las tres operaciones de conjuntos más comunes son

- La unión
- La intersección
- La diferencia.

11.1 Unión

La unión (**UNION**) combina los resultados de dos o más consultas en un solo conjunto, eliminando las filas duplicadas.

La sintaxis básica de la consulta de unión es la siguiente:

```
SELECT columnas
FROM tabla1

UNION

SELECT columnas
FROM tabla2;
```

Importante

Es importante tener en cuenta que las dos consultas deben tener la misma cantidad de columnas y los mismos tipos de datos para cada columna en el mismo orden.

Veamos un ejemplo:

Supongamos que tenemos dos tablas: "clientes" y "proveedores".

La tabla "clientes" tiene las siguientes columnas: "id_cliente", "nombre" y "direccion".

La tabla "proveedores" tiene las siguientes columnas: "id_proveedor", "nombre" y "ciudad".

Queremos combinar los nombres de clientes y proveedores en un solo conjunto de resultados.

La consulta quedaría así:

```
SELECT nombre, direccion, NULL AS ciudad
FROM clientes

UNION

SELECT nombre, NULL AS direccion, ciudad
FROM proveedores;
```

En esta consulta, la primera consulta selecciona el nombre y la dirección de los clientes, y utiliza "NULL" como valor para la columna "ciudad".

La segunda consulta selecciona el nombre de los proveedores y utiliza "NULL" como valor para la columna "direccion".

La cláusula **UNION** combina los resultados de ambas consultas en un solo conjunto de resultados, eliminando los duplicados automáticamente.

En este ejemplo, la primera consulta tiene tres columnas (nombre, dirección y ciudad con valor NULL) y la segunda consulta también tiene tres columnas (nombre con valor NULL, dirección con valor NULL y ciudad).

11.2 Intersección

La intersección (**INTERSECT**) devuelve las filas que se encuentran en ambos conjuntos de resultados de dos o más consultas (solo las coincidentes).

Definición

La cláusula **INTERSECT** en SQL se utiliza para obtener los registros que se encuentran en común entre dos conjuntos de resultados de consultas diferentes.

Esta cláusula devuelve únicamente los registros que coinciden en ambas consultas, eliminando los registros duplicados.

La sintaxis básica de la consulta de intersección es la siguiente:

```
SELECT columnas
FROM tabla1

INTERSECT

SELECT columnas
FROM tabla2;
```

Veamos un ejemplo:

Supongamos que tenemos dos tablas: "clientes" y "pedidos".

La tabla "clientes" tiene las siguientes columnas: "id_cliente", "nombre" y "pais".

La tabla "pedidos" tiene las siguientes columnas: "id_pedido", "id_cliente", "fecha" e "importe".

Queremos obtener los nombres de los clientes que han hecho pedidos en el mes de enero y también han hecho pedidos en el mes de febrero.

Para ello, podemos utilizar la cláusula INTERSECT de la siguiente manera:

```
SELECT nombre
FROM clientes
WHERE id_cliente IN

(
  SELECT id_cliente
  FROM pedidos
  WHERE MONTH(fecha) = 1

  INTERSECT

  SELECT id_cliente
  FROM pedidos
  WHERE MONTH(fecha) = 2
);
```

En esta consulta:

- La primera subconsulta selecciona los id de cliente que han hecho pedidos en enero
- la segunda subconsulta selecciona los id de cliente que han hecho pedidos en febrero.
- La cláusula INTERSECT combina ambos resultados y devuelve solo los id de cliente que se encuentran en ambos conjuntos de resultados.
- Finalmente, la consulta principal selecciona los nombres de los clientes cuyos id están presentes en el conjunto de resultados de la subconsulta.

Importante

Es importante tener en cuenta que las dos subconsultas deben tener la misma cantidad de columnas y los mismos tipos de datos para cada columna en el mismo orden.

Más info

La cláusula **INTERSECT** solo funciona en bases de datos que soportan SQL estándar.

Algunas bases de datos pueden utilizar otras cláusulas similares, como "**INTERSECT ALL**" o "**INTERSECT DISTINCT**".

11.3 Diferencia

La diferencia (**EXCEPT**) devuelve las filas que se encuentran en el conjunto de resultados de la primera consulta y no en el conjunto de resultados de la segunda consulta.

Es decir, se utiliza para devolver los registros que aparecen en el primer conjunto de resultados de una consulta, pero que no aparecen en el segundo conjunto de resultados de otra consulta.

Por tanto, solamente se devuelven los registros que son exclusivos del primer conjunto de resultados.

La sintaxis básica de la consulta de diferencia es la siguiente:

```
SELECT columnas
FROM tabla1

EXCEPT

SELECT columnas
FROM tabla2;
```

Importante

Es importante tener en cuenta que, para poder realizar operaciones de conjuntos, las consultas deben tener el mismo número de columnas y el mismo tipo de datos en cada columna.

Además, **las columnas en las que se basa la comparación deben tener el mismo nombre o alias en cada consulta.**

Vamos a ver un ejemplo:

Supongamos que tenemos dos tablas: "clientes" y "pedidos".

La tabla "clientes" tiene las siguientes columnas: "id_cliente", "nombre" y "pais".

La tabla "pedidos" tiene las siguientes columnas: "id_pedido", "id_cliente", "fecha" e "importe".

Queremos obtener los nombres de los clientes que **han hecho pedidos en el mes de enero, pero no han hecho pedidos en el mes de febrero.**

Para ello, podemos utilizar la cláusula EXCEPT de la siguiente manera:

```
SELECT nombre
FROM clientes
WHERE id_cliente IN

(
  SELECT id_cliente
  FROM pedidos
  WHERE MONTH(fecha) = 1

  EXCEPT

  SELECT id_cliente
  FROM pedidos
  WHERE MONTH(fecha) = 2
);
```

En esta consulta:

- La primera subconsulta selecciona los id de cliente que han hecho pedidos en enero
- La segunda subconsulta selecciona los id de cliente que han hecho pedidos en febrero.
- La cláusula EXCEPT combina ambos resultados y devuelve solo los id de cliente que se encuentran en el primer conjunto de resultados, pero no en el segundo.
- Finalmente, la consulta principal selecciona los nombres de los clientes cuyos id están presentes en el conjunto de resultados de la subconsulta.

Importante

Es importante tener en cuenta que, para poder realizar operaciones de conjuntos, las consultas deben tener el mismo número de columnas y el mismo tipo de datos en cada columna.

También hay que tener en cuenta que la cláusula **EXCEPT** solo funciona en bases de datos que soportan SQL estándar. Algunas bases de datos pueden utilizar otras cláusulas similares, como "MINUS" en Oracle

Veamos cómo se lleva a cabo el análisis de la actividad de clientes en BeautyGlow con operaciones de conjuntos

BeautyGlow desea analizar la actividad de sus clientes durante marzo de 2025, combinando datos de citas de servicios (Cita) y ventas de productos (Venta). Para ello utilizará las operaciones de conjuntos UNION, INTERSECT y EXCEPT.

Tablas de ejemplo:

Cita

id	cliente_id	servicio_id	fecha
1	101	5	2025-03-01
2	102	3	2025-03-05
3	101	2	2025-03-10
4	103	1	2025-03-12

Venta

id	cliente_id	producto_id	fecha
1	101	201	2025-03-03
2	104	202	2025-03-05
3	102	203	2025-03-10
4	105	204	2025-03-15

Cliente

id	nombre
101	Ana
102	Luis
103	Marta
104	Sara
105	Diego

Preguntas y soluciones

1. **Fechas de actividad en marzo (UNION)**

Obtener el conjunto de todas las fechas en las que hubo citas o ventas durante marzo de 2025.

```
SELECT fecha

FROM Cita

WHERE fecha BETWEEN '2025-03-01' AND '2025-03-31'


UNION


SELECT fecha

FROM Venta

WHERE fecha BETWEEN '2025-03-01' AND '2025-03-31'

ORDER BY fecha;
```

Resultado esperado:

fecha
2025-03-01
2025-03-03
2025-03-05
2025-03-10
2025-03-12
2025-03-15

UNION elimina automáticamente las fechas duplicadas.

2. **Clientes que hicieron tanto cita como compra (INTERSECT)**

Listar los nombres de los clientes que reservaron algún servicio y compraron algún producto en marzo de 2025.

```
SELECT nombre
FROM Cliente
WHERE id IN (
  SELECT cliente_id
  FROM Cita
  WHERE fecha BETWEEN '2025-03-01' AND '2025-03-31'

  INTERSECT

  SELECT cliente_id
  FROM Venta
  WHERE fecha BETWEEN '2025-03-01' AND '2025-03-31'
);
```

Subconsultas (cliente_id):

- Cita en marzo → {101, 102, 103}
- Venta en marzo → {101, 102, 104, 105}
- INTERSECT → {101, 102}

Resultado esperado:

nombre
Ana
Luis

3. Clientes con cita pero sin compra (EXCEPT)

Obtener los nombres de los clientes que hicieron una cita en marzo de 2025 pero no compraron ningún producto en ese mes.

```
SELECT nombre
FROM Cliente
WHERE id IN (
  SELECT cliente_id
  FROM Cita
  WHERE fecha BETWEEN '2025-03-01' AND '2025-03-31'

  EXCEPT

  SELECT cliente_id
  FROM Venta
  WHERE fecha BETWEEN '2025-03-01' AND '2025-03-31'
);
```

Subconsultas (cliente_id):

- Cita en marzo → {101, 102, 103}
- Venta en marzo → {101, 102, 104, 105}
- EXCEPT → {103}

Resultado esperado:

nombre
Marta

Interpretación de resultados

1. **Fechas de actividad**: Identifica días con cualquier tipo de interacción cliente–centro.
2. **Clientes "omnidireccionales"** (INTERSECT): Ana y Luis usaron ambos canales (servicios + venta), pudiendo ofrecerles promociones cruzadas.
3. **Clientes solo servicio** (EXCEPT): Marta reservó servicios, pero no compró productos; podría recibir campañas de upselling de productos de cuidado.

Con estas operaciones de conjuntos, BeautyGlow obtiene información clave para segmentar su base de clientes y diseñar estrategias de marketing dirigidas.

Tech Hub <FP> euroformac

Resumen

La sentencia SELECT se utiliza para extraer información de una base de datos. Se puede utilizar para seleccionar columnas específicas, filtrar registros y ordenar los resultados.

La selección, filtrado y ordenación de registros se realiza mediante la cláusula WHERE para filtrar registros basados en condiciones específicas y la cláusula ORDER BY para ordenar los resultados según una o más columnas.

Los operadores en SQL incluyen operadores de comparación, lógicos y aritméticos, que se utilizan en las condiciones de las consultas.

Las consultas de resumen permiten obtener información resumida sobre los datos, utilizando funciones de agregado como SUM, COUNT y AVG.

Las agrupaciones de registros permiten agrupar registros que tienen valores similares en una o más columnas. La cláusula HAVING se utiliza para filtrar los grupos.

Las composiciones internas se utilizan para combinar datos de dos o más tablas basadas en una condición específica.

Las composiciones externas se utilizan para obtener todos los registros de una tabla, incluso si no hay correspondencias en la otra tabla.

Las subconsultas son consultas anidadas dentro de otras consultas. Pueden devolver valores individuales, listas de valores, tuplas de valores o estar correlacionadas.

Las consultas jerárquicas se utilizan para representar datos jerárquicos, como árboles de categorías.

Las consultas de operaciones de conjuntos permiten combinar o comparar conjuntos de resultados de dos o más consultas mediante las operaciones de unión, intersección y diferencia.

Tech Hub <FP> euroformac

Glosario

- **Consultas para extraer información: la sentencia SELECT.**

Sentencia SELECT: Es una sentencia de SQL que se utiliza para seleccionar columnas específicas de una tabla y devolver los datos correspondientes.

FROM: Es una cláusula de SQL que se utiliza en conjunto con SELECT para especificar la tabla o tablas de las que se extraerán los datos.

- **Selección, filtrado y ordenación de registros.**

WHERE: Cláusula de SQL que se utiliza para filtrar los registros de una tabla basados en una o más condiciones.

Operadores de comparación: Son operadores que se utilizan en las condiciones de la cláusula WHERE para comparar valores, como "=", "<>", "<", "<=", ">", ">=".

Operadores lógicos: Son operadores que se utilizan para combinar múltiples condiciones en la cláusula WHERE, como "AND", "OR" y "NOT".

LIKE: Es un operador que se utiliza en la cláusula WHERE para buscar registros que contengan un patrón específico.

ORDER BY: Cláusula de SQL que se utiliza para ordenar los resultados de una consulta según una o más columnas.

ASC y DESC: Son palabras clave que se utilizan en la cláusula ORDER BY para indicar si se debe ordenar una columna en orden ascendente o descendente.

GROUP BY: Cláusula de SQL que se utiliza para agrupar los registros de una tabla según el valor de una o más columnas.

HAVING: Cláusula de SQL que se utiliza para filtrar los grupos devueltos por la cláusula GROUP BY.

- **Operadores (combinación, precedencia):**
  - **Operadores de comparación:**

=: Compara si dos valores son iguales.

!=: Compara si dos valores son diferentes.

< o >: Compara si un valor es menor o mayor que otro.

<= o >=: Compara si un valor es menor o igual o mayor o igual que otro.

- **Operadores lógicos:**

AND: Devuelve true si ambas expresiones son true.

OR: Devuelve true si al menos una de las expresiones es true.

NOT: Invierte el valor de una expresión.

- **Operadores aritméticos:**

+: Suma dos valores.

-: Resta dos valores.

*: Multiplica dos valores.

/: Divide dos valores.

% o MOD: Devuelve el resto de la división de dos valores.

^: Eleva un valor a una potencia.

Precedencia de operadores: Indica el orden en que se irán teniendo en cuenta los distintos operadores.

- **Consultas de resumen. Funciones de agregado**

Funciones de agregación: son funciones que se utilizan para realizar cálculos sobre grupos de registros, como sumas, promedios, conteos, etc.

Agrupaciones: son las columnas utilizadas para agrupar los registros y aplicar las funciones de agregación.

Cláusula GROUP BY: es una cláusula que se utiliza para agrupar los registros según una o varias columnas.

Cláusula HAVING: es una cláusula que se utiliza para filtrar los resultados de una consulta agregada que se han obtenido a través de la cláusula GROUP BY.

Funciones de suma: son funciones que se utilizan para sumar los valores de una columna en un grupo de registros. Algunas funciones de suma comunes son SUM, AVG y COUNT.

Funciones de promedio: son funciones que se utilizan para calcular el promedio de los valores de una columna en un grupo de registros. La función AVG es un ejemplo común.

Funciones de conteo: son funciones que se utilizan para contar el número de registros en un grupo de registros. La función COUNT es un ejemplo común.

Funciones de máximo y mínimo: son funciones que se utilizan para encontrar el valor máximo o mínimo de una columna en un grupo de registros. Las funciones MAX y MIN son ejemplos comunes.

Funciones de desviación estándar y varianza: son funciones que se utilizan para calcular la desviación estándar y la varianza de los valores de una columna en un grupo de registros. Las funciones STDEV y VAR son ejemplos comunes.

- **Agrupaciones de registros. Filtrado de las agrupaciones.**

Agrupación de registros: es el proceso de agrupar registros de una tabla en función de los valores de una o varias columnas. Se utiliza la cláusula GROUP BY para agrupar los registros.

Columnas de agrupación: son las columnas que se utilizan para agrupar los registros. Estas columnas deben aparecer en la cláusula GROUP BY.

Funciones de agregación: son las funciones que se utilizan para realizar cálculos sobre los valores de las columnas agrupadas. Estas funciones se aplican a grupos de registros en lugar de a registros individuales.

Filtrado de agrupaciones: es el proceso de aplicar un filtro a los resultados de una consulta que ha sido agrupada. Se utiliza la cláusula HAVING para filtrar los resultados de la consulta.

Cláusula HAVING: es una cláusula que se utiliza en las consultas agrupadas para filtrar los resultados basados en una condición. Esta cláusula se utiliza después de la cláusula GROUP BY.

Cláusula WHERE vs cláusula HAVING: la cláusula WHERE se utiliza para filtrar registros individuales, mientras que la cláusula HAVING se utiliza para filtrar grupos de registros.

- **Composiciones internas**

Composición interna: es una técnica que se utiliza para combinar registros de dos o más tablas basados en una condición en común. Se utilizan las cláusulas INNER JOIN o JOIN para realizar la composición interna.

Tablas combinadas: son las tablas que se combinan en una composición interna. Estas tablas deben tener una o más columnas en común para poder combinarse.

Columnas combinadas: son las columnas que se utilizan para combinar las tablas. Estas columnas deben tener los mismos nombres y tipos de datos en ambas tablas.

Cláusula INNER JOIN: es una cláusula que se utiliza para combinar dos o más tablas en una composición interna. Esta cláusula especifica las columnas que se utilizan para combinar las tablas.

Cláusula ON: es una cláusula que se utiliza en conjunto con la cláusula INNER JOIN para especificar la condición que se utiliza para combinar las tablas.

Cláusula JOIN: es una cláusula que se utiliza para combinar dos o más tablas en una composición interna. Esta cláusula es equivalente a INNER JOIN, pero no es tan comúnmente utilizada.

- **Composiciones externas**

Composiciones externas: son consultas que combinan registros de dos o más tablas, pero incluyen registros que no coinciden en ambas tablas. Existen dos tipos de composiciones externas: la composición externa izquierda (LEFT OUTER JOIN) y la composición externa derecha (RIGHT OUTER JOIN).

Composición externa izquierda (LEFT OUTER JOIN): esta consulta devuelve todos los registros de la tabla izquierda y los registros correspondientes de la tabla derecha. Si no hay registros coincidentes en la tabla derecha, se mostrarán valores NULL.

Composición externa derecha (RIGHT OUTER JOIN): esta consulta devuelve todos los registros de la tabla derecha y los registros correspondientes de la tabla izquierda. Si no hay registros coincidentes en la tabla izquierda, se mostrarán valores NULL.

Composición externa completa (FULL OUTER JOIN): es una combinación de la composición externa izquierda y la composición externa derecha, devuelve todos los registros de ambas tablas, incluidos los registros que no coinciden. Si no hay registros coincidentes en una tabla, se mostrarán valores NULL.

- **Subconsultas**

Subconsulta: es una consulta que se ejecuta dentro de otra consulta. Las subconsultas se utilizan para realizar operaciones más complejas que no se pueden realizar con una sola consulta.

Devolución de valores individuales: una subconsulta que devuelve un solo valor, que se puede utilizar en una expresión para realizar una operación adicional en la consulta principal.

Devolución de listas de valores: una subconsulta que devuelve una lista de valores y se utiliza en la cláusula IN o NOT IN para filtrar los registros de la consulta principal.

Devolución de tuplas de valores: una subconsulta que devuelve una o más tuplas de valores, que se pueden utilizar en la cláusula FROM de la consulta principal como una tabla.

Subconsulta correlacionada: es una subconsulta que depende de la consulta principal y se utiliza para filtrar los registros de la consulta principal. La subconsulta se ejecuta una vez para cada registro en la consulta principal, lo que significa que el resultado de la subconsulta depende del registro actual en la consulta principal.

- **Consultas jerárquicas.**

Consultas jerárquicas: son consultas que se utilizan para obtener información jerárquica de una tabla que tiene una relación jerárquica entre sus datos. Por lo general, se utilizan para consultar árboles de datos, como estructuras de organización, categorías de productos, etc.

Cláusula CONNECT BY: es una cláusula utilizada en consultas jerárquicas en Oracle que especifica cómo se relacionan los registros en la tabla jerárquica. Se utiliza para especificar el nivel jerárquico y la relación entre los registros en la tabla.

Pseudo-columna LEVEL: es una columna especial que se puede utilizar en consultas jerárquicas para indicar el nivel jerárquico de cada registro. Se puede utilizar en la cláusula SELECT para mostrar el nivel de cada registro.

Función PRIOR: es una función utilizada en consultas jerárquicas que se utiliza para hacer referencia al registro padre en una relación jerárquica. Se utiliza en la cláusula CONNECT BY para especificar la relación jerárquica entre los registros.

Consultas de operaciones de conjuntos: unión, intersección, diferencia.

Unión: Una operación que combina dos o más conjuntos de resultados en un solo conjunto de resultados que contiene todas las filas de cada conjunto, eliminando las duplicadas.

Intersección: Una operación que devuelve solo las filas que están presentes en ambos conjuntos de resultados.

Diferencia: Una operación que devuelve las filas que están en un conjunto de resultados, pero no en otro.

Tech Hub <FP> euroformac

Bibliografía

Medina Serrano, S. (2015). SQL Server 2014 Soluciones prácticas de administración [Formato tapa blanda]. Editorial Ra-Ma.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2001). Database System Concepts (4th ed.). McGraw-Hill.

Ullman, J. D. (1998). Principles of Databases and Knowledge Base Systems, Vol. I. Computer Science Press.

Tech Hub <FP> euroformac